PROYECTO DE AULA · TEORÍA DE LA COMPUTACIÓN

Avance 5
Diseño de la solución

Sistema de préstamo de herramientas basado en autómatas finitos y verificación visual

Documento de trabajo académico

Juan Diego Torres Mola

Ingeniería de Sistemas

Universidad de Pamplona

Docente: Mercy Lucero Gómez Rincón

Cúcuta, Norte de Santander

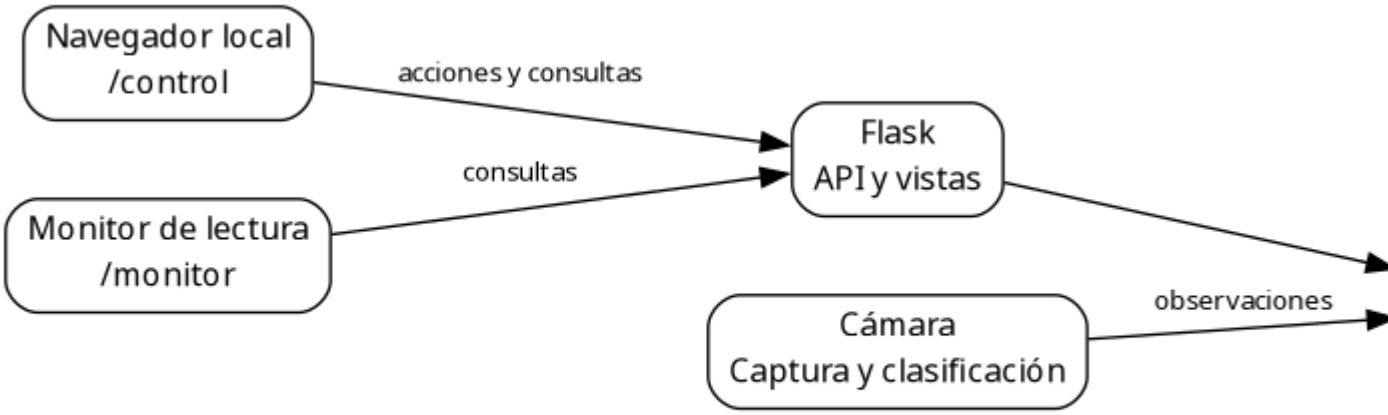
Avance 5 — VIII. Diseño de la solución

> ***Documento de sustentación técnica y de ingeniería.***
>
> *Este documento expone el diseño integral del* ***Sistema de préstamo de herramientas basado en autómatas finitos y verificación visual****, organizado en los cuatro componentes fundamentales establecidos por la guía académica (guía Avance 5):* ***A. Arquitectura del sistema****,* ***B. Diseño funcional****,* ***C. Diseño de datos*** *y* ***D. Diseño de interfaces****.*
>
> *El diseño responde a los requerimientos del Avance 4, describe la implementación del repositorio (`app/`, `simulator/`, `templates/`) y sustenta las decisiones de ingeniería adoptadas.*

A. Arquitectura del sistema

A.1 Estructura general de la solución en capas

La solución adopta una arquitectura desacoplada por capas con una **única autoridad de estado centralizada**, ejecutándose en un entorno de red de área local (LAN) o estación de trabajo independiente. El diseño separa limpiamente la interfaz de usuario, la exposición de servicios web, la lógica de control formal, el subsistema de visión por computadora, la adaptación de sensores y la persistencia atómica.



Arquitectura del módulo visual

Figura A1. Arquitectura del prototipo visual. El servidor recibe acciones locales y consultas del monitor. El motor valida reglas y guarda registros; la cámara aporta observaciones y el adaptador representa puerta y presencia simuladas. El simulador del puerto 8001 es independiente y mantiene estado en memoria.

A.2 Distinción fundamental de los tres entornos del proyecto

Para evitar confusiones conceptuales durante la sustentación académica, el diseño delimita expresamente tres componentes técnicos diferenciados:

1. **Prototipo principal con cámara y hardware simulado (Puerto 8000):**

Es el prototipo principal (`app/`, `run.py`). Ejecuta el motor determinista con persistencia en `data/prestamos.sqlite`, soporta la captura de video en vivo mediante cámara web o DroidCam, evalúa la ventana temporal de evidencia visual mediante `CameraManager`, aplica el token de seguridad para el Centro de Control local y sirve la vista sincronizada de monitoreo.

1. **Simulador didáctico independiente (Puerto 8001):**

Es un entorno puramente educativo (`simulator/`, `run_simulador.py`) desarrollado para permitir la experimentación interactiva de los autómatas sin requerir una cámara conectada, sin cargar modelos neuronales y sin alterar la base de datos de préstamos. Administra su estado exclusivamente en memoria RAM y cuenta con controles 3D y guiones pregrabados de demostración.

1. **Hardware simulado (`SimulatedHardware`) vs. Circuito electromecánico:**

En la fase actual del prototipo, las señales físicas de apertura/cierre de la compuerta y el sensor de presencia simulado se gestionan mediante el adaptador `SimulatedHardware` (`app/hardware.py`). Una orden de apertura emitida por el motor solicita la acción al adaptador, pero la transición a puerta abierta no ocurre hasta que el sensor simulado emite el evento de conmutación. Esto replica la arquitectura desacoplada prevista para una futura integración física, cuyos componentes y protocolo deben definirse y validarse por separado.

A.3 Mecanismos de comunicación y seguridad

- **API REST Local:** El cliente web se comunica mediante peticiones asíncronas JSON. La consulta de estado (`GET /api/estado?desde=seq`) utiliza paginación por cursores para recibir únicamente los nuevos eventos ocurridos desde la última revisión.
- **Seguridad en el Centro de Control:** Las operaciones de mutación (`POST /api/acciones`) exigen que la petición provenga de una dirección IP de bucle invertido (*loopback* `127.0.0.1` o `::1`), que coincida la cabecera `Origin` y que incluya el token criptográfico `X-Control-Token` generado de forma aleatoria con 32 bytes (`secrets.token_urlsafe(32)`) y validado mediante comparación de tiempo constante (`hmac.compare_digest`).
- **Aislamiento del Monitor (`/monitor`):** La vista para tabletas y observadores no recibe el token de control. Si un usuario intenta enviar una petición `POST` desde un dispositivo remoto en la red local, el servidor rechaza la solicitud con código HTTP 403 Forbidden.
- **Idempotencia de peticiones:** Cada acción enviada desde la interfaz contiene un identificador único de petición (`id`). Si una petición se retransmite por inestabilidad de la red, el motor localiza el identificador en la tabla `requests` y entrega el resultado previo almacenado sin reejecutar la lógica ni duplicar registros.

A.4 Diagrama arquitectónico detallado

El diagrama formal en Mermaid se encuentra disponible en `docs/avances/diagramas/arquitectura.mmd` y se renderiza a continuación:

Diagrama editable disponible en la carpeta `docs/avances/diagramas`.

B. Diseño funcional

B.1 Módulos del sistema y responsabilidades

El sistema se estructura en siete módulos funcionales claramente delimitados:

Módulo Funcional	Componente de Software	Responsabilidad Principal	Requerimientos Asociados
1. Control de Inventario	`app/automata.py` (`INVENTORY`), `app/engine.py`	Gestionar el ciclo administrativo del martillo a través de sus 5 estados formales (`Disponible`, `Reservada`, `Prestada`, `En revisión`, `Fuera de servicio`), rechazando cualquier evento no admitido.	RF01, RF02, RF03, RF04, RF05, RF06, RF07; RNF01, RNF02
2. Control de Operación	`app/automata.py` (`OPERATION`), `app/engine.py`	Coordinar la secuencia de apertura física, espera de interacción, cierre, verificación sensorial y tratamiento de incidencias en 8 estados operativos.	RF02, RF03, RF04, RF05, RF07, RF10; RNF01, RNF04
3. Captura y Visión Artificial	`app/camera.py` (`CameraManager`)	Capturar flujo de video, previsualizar en JPEG, recortar ROI, clasificar imágenes con YOLO11n-cls y gestionar almacenamiento de muestras por sesión sin fuga entre conjuntos.	RF02, RF03, RF04, RF05; RNF06, RNF07
4. Gestión de Préstamos y Usuarios	`app/engine.py` (`dispatch`, `_apply`)	Asociar préstamos a códigos de usuario demostrativos, comprobar titularidad en devoluciones y mantener los préstamos en SQLite. El índice único parcial impide dos préstamos activos para la misma herramienta.	RF02, RF03, RF04, RF05, RF08; RNF02, RNF04
5. Revisión Técnica Humana	`app/engine.py` (`action in ('aprobar', 'danada', 'reparada')`)	Permitir que el encargado registre su decisión tras revisar la herramienta. La aprobación devuelve el inventario a Disponible; el daño lo deja Fuera de servicio.	RF06
6. Monitor y Auditoría	`app/web.py` (`monitor`, `history`), `templates/portal.html`	Exponer la vista de solo lectura `/monitor` para tablets y suministrar la descarga del registro secuencial en formato CSV estructurado.	RF01, RF08, RF09; RNF03, RNF05
7. Simulación Didáctica Independiente	`simulator/engine.py`, `simulator/web.py`	Permitir la exploración visual e interactiva de los autómatas con escena 3D y demostraciones automáticas sin alterar los datos del sistema real.	RF01, RF07; RNF01 (Propósito educativo y de sustentación)

B.2 Modelado formal mediante autómatas finitos deterministas

Autómata del ciclo de inventario

El ciclo de vida administrativo de la herramienta se modela como un AFD determinista formal:

$M_{inv} = (Q_{inv}, \Sigma_{inv}, \delta_{inv}, q_0, F_{inv})$

- $Q_{inv} = \{\text{Disponible}, \text{Reservada}, \text{Prestada}, \text{En revisión}, \text{Fuera de servicio}\}$

- `q_0 = \text{Disponible}`
- `F_{inv} = \{\text{Disponible}\}` (único estado donde el ciclo se encuentra cerrado y en descanso)
- `\Sigma_{inv} = \{\text{solicitud\_verificada}, \text{retiro\_verificado}, \text{cancelacion\_verificada}, \text{devolucion\_verificada}, \text{aprobacion}, \text{dano}, \text{reparacion}\}`

Tabla determinista de transiciones `\delta_{inv}`:

Estado Origen (q)	**Evento (`\sigma`)**	**Estado Destino (`\delta(q, \sigma)`)**	**Condición de Guarda Requerida**
`Disponible`	`solicitud_verificada`	`Reservada`	Operación inactiva, puerta cerrada, presencia=True y visión="martillo" estable.
`Reservada`	`retiro_verificado`	`Prestada`	Cierre confirmado, presencia=False y visión="vacio" estable post-cierre.
`Reservada`	`cancelacion_verificada`	`Disponible`	Cancelación solicitada, puerta cerrada, presencia=True y visión="martillo".
`Prestada`	`devolucion_verificada`	`En revisión`	Cierre confirmado, presencia=True y visión="martillo" estable post-cierre.
`En revisión`	`aprobacion`	`Disponible`	Código de encargado (`ADMIN`), puerta cerrada, presencia=True y visión="martillo".
`En revisión`	`dano`	`Fuera de servicio`	Código de encargado (`ADMIN`) reportando avería o desgaste visible.
`Fuera de servicio`	`reparacion`	`En revisión`	Código de encargado (`ADMIN`) certificando finalización de arreglo técnico.

El diagrama en Mermaid se almacena en `docs/avances/diagramas/automata_inventario.mmd` y se presenta a continuación:

Diagrama editable disponible en la carpeta `docs/avances/diagramas`.

Autómata del control de operación física

Coordina la interacción electromecánica y sensorial en la casilla física:

`$M_{op} = (Q_{op}, \Sigma_{op}, \delta_{op}, \text{Inactiva}, \{\text{Inactiva}\})$`

- `Q_{op} = \{\text{Inactiva}, \text{Abriendo}, \text{Esperando retiro}, \text{Esperando depósito}, \text{Cerrando}, \text{Verificando}, \text{Completada}, \text{Incidencia}\}`
- Modos operativos asociados: `\text{modo} \in \{\text{prestamo}, \text{devolucion}, \text{cancelacion}\}`

El diagrama se almacena en docs/avances/diagramas/automata_operacion.mmd y se renderiza a continuación:

Diagrama editable disponible en la carpeta `docs/avances/diagramas`.

Reconocedor de trazas (AFD Académico en software)

Para la comprobación formal del comportamiento del inventario, la función `recognize(events)` en `app/automata.py` implementa el AFD completo:

$M_{rec} = (Q_{inv} \cup \{\text{Error}\}, \Sigma_{inv}, \delta_{rec}, \text{Disponible}, \{\text{Disponible}\})$

Cualquier evento que no coincida con la tabla determinista envía la máquina al estado de sumidero absorbente `\text{Error}`, desde el cual cualquier evento subsiguiente permanece en `\text{Error}`. Esto permite certificar de manera automatizada si una traza histórica ejecutada constituye una secuencia regular válida y terminada.

C. Diseño de datos

C.1 Modelo conceptual Entidad-Relación (ER)

El modelo conceptual captura las entidades requeridas para sustentar el préstamo controlado, su historial persistente y la persistencia de estados.

Diagrama editable disponible en la carpeta `docs/avances/diagramas`.

El diagrama original se conserva en docs/avances/diagramas/modelo_er.mmd.

C.2 Modelo relacional físico y esquema SQLite

El prototipo utiliza SQLite como motor relacional empotrado. El esquema físico generado en `app/engine.py` (líneas 41–50) se documenta a continuación con estricta fidelidad técnica:

```
PRAGMA journal_mode=WAL;

CREATE TABLE IF NOT EXISTS state (
    id INTEGER PRIMARY KEY CHECK(id=1),
    value TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS users (
    code TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    role TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS loans (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_code TEXT NOT NULL,
    tool TEXT NOT NULL,
    borrowed_at TEXT NOT NULL,
    returned_at TEXT,
    simulated_hardware INTEGER NOT NULL
);

CREATE UNIQUE INDEX IF NOT EXISTS one_active_loan
ON loans(tool) WHERE returned_at IS NULL;

CREATE TABLE IF NOT EXISTS events (
    seq INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    time TEXT NOT NULL,
    machine TEXT,
    event TEXT NOT NULL,
    previous TEXT,
    current TEXT,
    source TEXT NOT NULL,
    accepted INTEGER NOT NULL,
    message TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS requests (
    id TEXT PRIMARY KEY,
    payload TEXT NOT NULL,
    result TEXT NOT NULL
);
```

Diccionario de datos y especificación de tablas

Tabla: `state`

Columna	`id`
Tipo SQLite	`INTEGER`
Restricción	`PRIMARY KEY CHECK(id=1)`
Propósito y regla de negocio	Permite como máximo una fila, con identificador 1. La aplicación crea o recupera esa fila al iniciar.

Tabla: `state`

Columna	`value`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Objeto JSON estructurado con el estado de ambos autómatas, variables de compuerta, presencia y mensajes.

Tabla: `users`

Columna	`code`
Tipo SQLite	`TEXT`
Restricción	`PRIMARY KEY`
Propósito y regla de negocio	Código alfanumérico identificador (`1001`, `1002`, `ADMIN`).

Tabla: `users`

Columna	`name`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`

Propósito y regla de negocio	Nombre descriptivo del actor.

Tabla: `users`

Columna	`role`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Rol de seguridad: `'usuario'` para estudiantes o `'encargado'` para el administrador.

Tabla: `loans`

Columna	`id`
Tipo SQLite	`INTEGER`
Restricción	`PRIMARY KEY AUTOINCREMENT`
Propósito y regla de negocio	Identificador autoincremental de préstamo.

Tabla: `loans`

Columna	`user_code`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Código del usuario responsable. **Relación lógica** con `users.code` (ver aclaración abajo).

Tabla: `loans`

Columna	`tool`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Herramienta prestada (`'martillo'`).

Tabla: `loans`

Columna	`borrowed_at`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`

Propósito y regla de negocio	Marca temporal ISO 8601 UTC en milisegundos (`utc()`) en que se verificó el retiro.

Tabla: `loans`

Columna	`returned_at`
Tipo SQLite	`TEXT`
Restricción	`NULL`
Propósito y regla de negocio	Marca temporal ISO UTC de devolución verificada. Si es `NULL`, el préstamo permanece **activo**.

Tabla: `loans`

Columna	`simulated_hardware`
Tipo SQLite	`INTEGER`
Restricción	`NOT NULL`
Propósito y regla de negocio	Entero booleano (1 si se utilizó hardware simulado, 0 para hardware real).

Tabla: `events`

Columna	`seq`
Tipo SQLite	`INTEGER`
Restricción	`PRIMARY KEY AUTOINCREMENT`
Propósito y regla de negocio	Número secuencial estrictamente creciente para paginación incremental de auditoría.

Tabla: `events`

Columna	`time`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Marca temporal ISO UTC del suceso.

Tabla: `events`

Columna	`machine`
Tipo SQLite	`TEXT`
Restricción	`NULL`

Propósito y regla de negocio	Nombre del autómata afectado (`'inventario'`, `'operacion'` o `NULL` si es general).

Tabla: `events`

Columna	`event`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Identificador del evento ejecutado o intentado.

Tabla: `events`

Columna	`previous`
Tipo SQLite	`TEXT`
Restricción	`NULL`
Propósito y regla de negocio	Estado previo a la transición.

Tabla: `events`

Columna	`current`
Tipo SQLite	`TEXT`
Restricción	`NULL`
Propósito y regla de negocio	Estado alcanzado tras la transición.

Tabla: `events`

Columna	`source`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Origen: `'usuario'`, `'camara'`, `'simulador'` o `'sistema'`.

Tabla: `events`

Columna	`accepted`
Tipo SQLite	`INTEGER`
Restricción	`NOT NULL`

Propósito y regla de negocio	1 si la transición fue aceptada; 0 si la acción fue rechazada.

Tabla: `events`

Columna	`message`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Explicación textual del cambio o justificación del rechazo.

Tabla: `requests`

Columna	`id`
Tipo SQLite	`TEXT`
Restricción	`PRIMARY KEY`
Propósito y regla de negocio	Identificador UUID enviado por el cliente para control de idempotencia.

Tabla: `requests`

Columna	`payload`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Carga JSON canónica de la petición.

Tabla: `requests`

Columna	`result`
Tipo SQLite	`TEXT`
Restricción	`NOT NULL`
Propósito y regla de negocio	Respuesta JSON almacenada para reenvío directo en peticiones repetidas.

[!IMPORTANT]

Fidelidad técnica: Llaves Primarias y Relaciones Lógicas sin invención de FK:

*En el esquema DDL real de SQLite, la columna `loans.user_code` **no cuenta con una cláusula formal de clave foránea** (`FOREIGN KEY ... REFERENCES users(code)`). La integridad referencial entre préstamos y usuarios se valida de forma imperativa en la lógica de negocio de Python a través del método*

> *`Engine._user(code)` (`app/engine.py`, línea 98), el cual comprueba la existencia del registro en `users` antes de autorizar cualquier préstamo. En estricto apego a la honestidad técnica, este vínculo se modela como una **relación lógica de dominio**, sin falsificar restricciones sintácticas que no existen en el código SQLite real.*

Diagrama relacional físico

El diagrama relacional se conserva en `docs/avances/diagramas/modelo_relacional.mmd` y se renderiza a continuación:

Diagrama editable disponible en la carpeta `docs/avances/diagramas`.

D. Diseño de interfaces

El diseño de interfaces provee vistas diferenciadas para la operación autorizada en la estación local, la supervisión de solo lectura en pantallas remotas y la simulación didáctica.

Fig. 1. Centro de control local (`templates/portal.html` con `mode="control"`)

Pantalla principal diseñada para el computador de la estación de trabajo local (`http://127.0.0.1:8000/control`). Permite al operador y a los usuarios interactuar con el flujo del prototipo bajo autorización estricta:

- **Selector de usuario demostrativo:** Permite elegir entre los códigos `1001`, `1002` o `ADMIN`.
- **Panel de acciones operativas:** Despliega dinámicamente los botones habilitados según el estado de la máquina: «Solicitar martillo», «Cerrar», «Finalizar», «Iniciar devolución» o «Cancelar». Los botones no aplicables se deshabilitan para evitar confusiones.
- **Sección de revisión técnica del encargado:** Muestra los botones de supervisión humana («Aprobar», «Registrar daño», «Reparada»), protegidos por validación del rol `encargado`.
- **Monitor de visión y hardware:** Exhibe el cuadro de previsualización JPEG de la cámara con el recuadro de la región de interés (ROI), el estado de conexión del dispositivo, la clase predicha con su confianza porcentual, y los interruptores para conmutar las señales de hardware simulado (puerta y sensor de presencia).
- **Indicador luminoso virtual (LED):** Semáforo visual que refleja el estado de la operación: verde (en proceso), rojo fijo (prestada/no disponible), rojo intermitente (incidencia) o apagado (disponible).

AUTÓMATAS
LABORATORIO DE HERRAMIENTAS

Control · Monitor ↗ · ● Conectado

TEORÍA DE LA COMPUTACIÓN / PROTOTIPO 01

Centro de control

Un compartimiento. Dos autómatas. Cada cambio tiene una causa.

● Puerta y sensores simulados
Cámara y reconocimiento reales

01 / HERRAMIENTA
Disponible
Martillo · compartimiento 01

02 / OPERACIÓN
Inactiva
Sin operación activa

03 / VISIÓN ARTIFICIAL
Sin evidencia
Modelo o cámara pendientes

04 / REGISTRO
5 eventos
Sincronizado 11:13:30 a. m.

ⓘ Coloca el martillo y configura la cámara. Los sensores son simulados.

OPERADOR
Préstamo y devolución
01

Código de usuario
1001 — Estudiante 1
Códigos de demostración: 1001, 1002 y ADMIN para revisión.

Solicitar martillo ↗ | Iniciar devolución ↙

Cerrar y verificar | Cancelar reserva | Reintentar | Finalizar operación

Sin préstamo activo.

▸ **Revisión del encargado**

ADAPTADOR SIMULADO
Compartimiento virtual
Sin electrónica



Puerta cerrada
LED apagado
Ilustración del sensor.
No es una detección visual.

Confirmar puerta abierta | Confirmar puerta cerrada

Sensor de presencia
Indica si hay un objeto en el soporte ☐

Sensor conectado
Desactívalo para ensayar un fallo ☑

VISIÓN ARTIFICIAL
Una imagen, una comprobación
Modelo pendiente



Sin iniciar

Índice de cámara en Windows
Cámara 0
DroidCam puede ser 0, 1 u otro índice. Si aparece otra cámara, detén y cambia el índice.

▸ **Zona observada (ROI)**

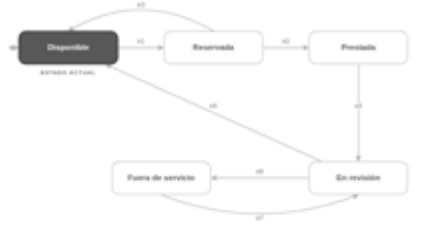
Iniciar cámara | Detener

El modelo se entrena con tus herramientas.
Sin modelo puedes capturar imágenes. El sistema no autoriza préstamos hasta recibir reconocimiento real.
Instalación: `.\instalar_vision.ps1`
Guía: `docs/RECONOCIMIENTO.md`

▸ **Preparar las imágenes de entrenamiento**

EL MODELO QUE CONTROLA EL SISTEMA
Estados y transiciones
■ Estado actual ┅ Última transición ╌╌ Incidencia

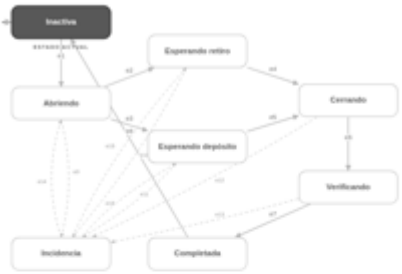
Ciclo de la herramienta
5 estados



Estado inicial: Disponible. Todavía no hay transiciones.

▸ **Alfabeto y tabla de transición**

Control de la operación
8 estados



Estado inicial: Inactiva. Todavía no hay transiciones.

▸ **Alfabeto y tabla de transición**

TRAZABILIDAD
Cada evento cuenta
Exportar CSV ↓

Mostrar: Todos los eventos — 5 eventos recibidos

# / HORA	ORIGEN	EVENTO	RESULTADO
#5 11:11:02 a. m.	USUARIO	**solicitar** El sensor simulado no indica presencia.	Rechazado
#4 12:51:14 p. m.	USUARIO	**solicitar** El sensor simulado no indica presencia.	Rechazado
#3 12:48:09 p. m.	USUARIO	**solicitar** El sensor simulado no indica presencia.	Rechazado
#2 12:47:42 p. m.	USUARIO	**solicitar** El sensor simulado no indica presencia.	Rechazado
#1 12:47:23 p. m.	SISTEMA	**inicializacion** Prototipo iniciado: sensores y puerta simulados.	Registrado

Fig. 1. Prototipo del Centro de Control local en interfaz de escritorio (`mode="control"`), mostrando botones de acción, estado de sensores simulados y visor de cámara.

Fig. 2. Monitor de visualización de autómatas (`templates/portal.html` con `mode="monitor"`)

Vista de solo lectura orientada a una segunda pantalla, tableta de supervisión o proyector de exposición (`http://IP_DEL_COMPUTADOR:8000/monitor`).

- **Diagramas de estados SVG interactivos:** Presenta los dos autómatas en pantalla completa (lado a lado en orientación horizontal o apilados en orientación vertical). Los nodos correspondientes al estado corriente se resaltan con iluminación activa y las transiciones ejecutadas animan su recorrido.
- **Transparencia operativa y seguridad:** La vista se actualiza automáticamente cada 500 ms mediante consultas incrementales a `/api/estado`. No incluye ningún formulario, botón de control ni el token criptográfico de escritura.
- **Panel de telemetría y conexión:** Muestra el indicador de sincronización en tiempo real, la hora de la última sincronización, el último mensaje descriptivo del motor y la indicación explícita de hardware simulado.

AUTÓMATAS
LABORATORIO DE HERRAMIENTAS

Solo lectura | Ampliar diagramas | Cone

TEORÍA DE LA COMPUTACIÓN / PROTOTIPO 01

Los autómatas, en vivo

Sigue el estado de la herramienta y la operación, evento por evento.

Puerta y sensores simul
Cámara y reconocimiento rea

01 / HERRAMIENTA
Disponible
Martillo - compartimiento 01

02 / OPERACIÓN
Inactiva
Sin operación activa

03 / VISIÓN ARTIFICIAL
Sin evidencia
Modelo o cámara pendientes

04 / REGISTRO
5 eventos
Sincronizado 11:13:30 a. m.

Coloca el martillo y configura la cámara. Los sensores son simulados.

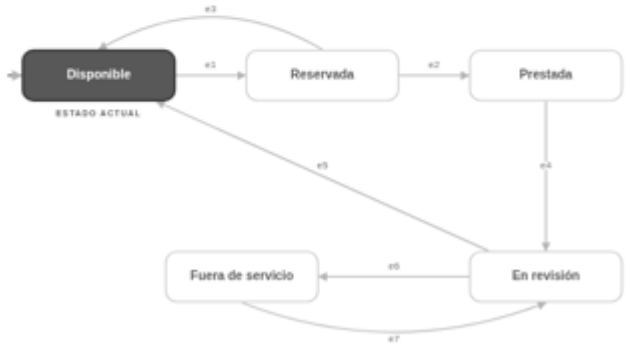
EL MODELO QUE CONTROLA EL SISTEMA

Estados y transiciones

Estado actual | Última transición

Ciclo de la herramienta

5 estados

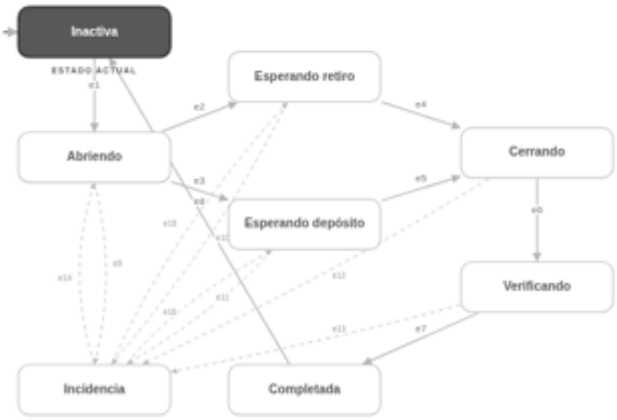


Estado inicial: Disponible. Todavía no hay transiciones.

▸ Alfabeto y tabla de transición

Control de la operación

8 esta



Estado inicial: Inactiva. Todavía no hay transiciones.

▸ Alfabeto y tabla de transición

TRAZABILIDAD

Cada evento cuenta

Exportar CSV

Mostrar: Todos los eventos

5 eventos recibi

# / HORA	ORIGEN	EVENTO	RESULTADO
#5 11:11:02 a. m.	USUARIO	**solicitar** El sensor simulado no indica presencia.	Rechazado
#4 12:51:14 p. m.	USUARIO	**solicitar** El sensor simulado no indica presencia.	Rechazado
#3 12:48:09 p. m.	USUARIO	**solicitar** El sensor simulado no indica presencia.	Rechazado
#2 12:47:42 p. m.	USUARIO	**solicitar** El sensor simulado no indica presencia.	Rechazado
#1 12:47:19 p. m.	SISTEMA	**inicializacion** Prototipo iniciado; sensores y puerta simulados.	Registrado

La vista muestra los 200 eventos más recientes del filtro. El registro completo se conserva en SQLite y en la exportación CSV.

Fig. 2. Monitor de supervisión en tiempo real para tableta (`mode="monitor"`), mostrando los autómatas de inventario y operación coordinados y el mensaje de estado del sistema.

Fig. 3. Simulador didáctico 3D interactivo (`templates/simulation.html`)

Módulo complementario e independiente para la sustentación pedagógica y experimentación de contingencias (`http://127.0.0.1:8001/simulador`):

- **Compartimiento 3D virtual:** Renderizado en perspectiva con controles orbitales de giro, mostrando la herramienta en el interior, la apertura/cierre animada de la compuerta y la presencia de objetos.
- **Modos de ejecución dual:** Permite avanzar paso a paso manualmente o reproducir de manera automática cualquiera de los cinco escenarios didácticos preconfigurados:

1. *Ciclo completo:* Reserva, retiro, devolución y aprobación exitosas.
2. *Objeto incorrecto:* Intento de cierre depositando un alicate, rechazo inmediato por discrepancia visual y corrección posterior con martillo.
3. *Fallo y recuperación:* Desconexión forzada del sensor durante la verificación, entrada en estado `Incidencia` y procedimiento de reintento.
4. *Daño y reparación:* Flujo completo de inspección humana: `En revisión` → `Fuera de servicio` → `En revisión` → `Disponible`.
5. *Cancelar reserva:* Cancelación del retiro con el martillo en la casilla, liberando la herramienta a `Disponible` sin crear préstamo.

- **Autonomía total:** No interactúa con la base de datos `prestamos.sqlite` ni requiere cámara web, permitiendo demostrar la lógica sin depender de la iluminación de una cámara.

Correspondencia y trazabilidad con el Avance 4

La tabla siguiente relaciona componentes del diseño con requisitos definidos en la Fase de Análisis:

Elemento de Análisis (Avance 4)	Concreción en el Diseño (Avance 5)	Justificación de Coherencia
Actor: Usuario / Estudiante	Centro de control (`portal.html`), roles en `users`, métodos `solicitar`, `devolver`, `cerrar`.	Permite al estudiante retirar y devolver con validación previa y posterior.
Actor: Encargado de Laboratorio	Sección de revisión técnica en panel de control, acciones `aprobar`, `danada`, `reparada`.	Permite comprobar que ninguna herramienta reingrese al inventario sin inspección humana.
Actor: Visión y Sensores	`CameraManager` con YOLO11n-cls y `SimulatedHardware`.	Convierte observaciones físicas y ópticas en entradas discretas para el motor.
RF01 (Visualización de estados)	Endpoint `/api/estado`, monitor `/monitor` y diagramas SVG interactivos.	Presenta ambos autómatas con refresco de 500 ms y telemetría de conexión.
RF02, RF03	Acciones `solicitar`, `abrir_retiro`,	Bloquea el registro de retiros que no

Elemento de Análisis (Avance 4)	Concreción en el Diseño (Avance 5)	Justificación de Coherencia
(Préstamo verificado)	`cerrar`, verificación visual post-cierre e inserción en tabla `loans`.	cumplan las reglas y exige que el préstamo solo se cree tras verificar casilla vacía.
RF04, RF05 (Devolución y revisión)	Acción `devolver`, validación de responsable en `loans`, verificación post-cierre de martillo y transición a `En revisión`.	Cierra el registro del préstamo pero bloquea la disponibilidad hasta la aprobación del encargado.
RF06 (Inspección técnica)	Métodos `_apply` para `aprobar`, `danada`, `reparada` con verificación `admin=True`.	Soporta el tratamiento de herramientas deterioradas y su retorno seguro.
RF07 (Rechazo determinista)	Excepción `Rejected`, reversión de estado (`deepcopy`) y registro en `events` con `accepted = 0`.	Mantiene la integridad formal del autómata ante secuencias o datos inválidos.
RF08 (Historial persistente)	Tabla relacional `events` y descarga CSV vía `/api/historial.csv`.	Proporciona trazabilidad completa para auditoría académica y técnica.
RNF02 (Unicidad de préstamos)	Índice único parcial `one_active_loan` en tabla `loans`.	Impide dos préstamos simultáneos del mismo martillo a nivel de motor de datos.
RNF03 (Seguridad local)	Filtro loopback en Flask, cabeceras `Origin`/`Sec-Fetch-Site` y token aleatorio `X-Control-Token`.	Protege las funciones de control de operaciones frente a peticiones indebidas de la red.

Límites técnicos, evidencia actual y verificaciones pendientes

1. Estado de verificación de software

- **Suite de pruebas automatizadas:** Al 5 de noviembre de 2026, la suite completa de pruebas consta de **47 pruebas automatizadas aprobadas en 0,62 segundos** (`.venv/bin/python -m pytest -q`).
- **Naturaleza de la evidencia:** Estas pruebas corresponden a **observaciones controladas en memoria**, fixtures simulados y escenarios sintéticos que comprueban la lógica de los autómatas, la persistencia en SQLite, el aislamiento de permisos y la sincronización. **No constituyen una medición de precisión en condiciones físicas reales de laboratorio**.

2. Dispositivos físicos y hardware simulado

- Las compuertas y sensores de presencia simulados están gobernados por software (`SimulatedHardware`). No existe un circuito Arduino ni actuador electromecánico instalado en la casilla en esta etapa. Por ello, las devoluciones se registran formalmente como *verificación visual con hardware simulado*, sin certificar la existencia de una cerradura electromecánica física.

3. Modelo de visión artificial y dataset propio

- Al momento de esta documentación, la carpeta `dataset/` no contiene imágenes recolectadas y el archivo `models/herramientas.pt` está ausente en el repositorio.

- El prototipo cuenta con la infraestructura completa para captura guiada (`CameraManager.capture`) con control de sesiones (`sessions.json`) para prevenir fuga de datos entre `train`, `val` y `test`, y con scripts de entrenamiento (`tools/train_model.py`) y evaluación (`tools/evaluate_model.py`).
- **Pendiente indispensable para la demostración con cámara:**

1. Conectar cámara web o celular mediante DroidCam y fijar iluminación y encuadre.
2. Capturar muestras fotográficas para las cinco clases (`martillo`, `destornillador`, `alicate`, `vacio`, `otros`).
3. Ejecutar el entrenamiento del clasificador YOLO11n-cls.
4. Generar la matriz de confusión y medir exactitud y tasa de falsos positivos en el conjunto de prueba independiente.
5. Ejecutar los diez ciclos de validación con cámara real documentando los tiempos reales.

No se asignan cifras de precisión o exactitud hasta que el equipo realice estas mediciones en hardware real.

Uso ético y responsable de la inteligencia artificial

Conforme a las disposiciones académicas del curso y las directrices de la guía del Avance 5:

- La inteligencia artificial se empleó exclusivamente como asistente técnico en la estructuración de la arquitectura, redacción de especificaciones de ingeniería y diseño de sintaxis de diagramas Mermaid.
- Cada componente, tabla relacional, autómata y flujo fue confrontado con el código fuente del proyecto (`app/engine.py`, `app/automata.py`, `app/web.py`, `app/camera.py`, `simulator/engine.py`), corrigiendo cualquier desviación previa (como la denominación del modo de vista en `web.py`, que es `mode="monitor"` y no `tablet`).
- Se respetó la fidelidad estricta del esquema SQLite, documentando la relación lógica entre préstamos y usuarios sin inventar restricciones foráneas no implementadas.
- No se introdujeron datos privados, sensibles ni de terceros.
- El equipo estudiantil debe revisar y comprender el diseño y validar su correspondencia con el contexto antes de la sustentación. Esta redacción no acredita una revisión humana ya realizada.

Figuras y alcance de las evidencias de interfaz

Las capturas incluidas proceden de las pruebas históricas de interfaz descritas en `docs/VALIDACION.md`; no muestran entrenamiento de un modelo propio ni una prueba de custodia física. La galería de figuras contiene versiones PNG, SVG y fuentes DOT editables de arquitectura, autómatas y datos. El modelo conceptual describe entidades del dominio; el modelo relacional utiliza los tipos TEXT e INTEGER del esquema SQLite real.